

華中科技大學

机器学习作业报告

专 业： 数据科学与大数据技术

班 级： 2402

学 号： U202414772

姓 名： 庄梓泓

电 话： 15387100663

邮 箱： u202414772@hust.edu.cn

基于深度学习的 CIFAR-10 图像分类研究

摘要

本文基于 CIFAR-10 数据集开展图像分类实验，围绕小尺寸彩色图像的多类别识别任务，分别构建并训练多层感知机（MLP）、普通卷积神经网络（CNN）和 ResNet18 三种模型。实验首先对 CIFAR-10 数据集进行读取、张量转换、归一化和数据增强处理，然后在统一训练流程下完成模型训练、测试评估和结果保存。为了全面比较不同模型结构的分类性能，本文采用准确率、损失曲线、分类报告和混淆矩阵等指标对实验结果进行分析。

实验结果表明，MLP 由于将图像直接展平成一维向量，难以充分利用像素之间的空间结构信息，因此分类效果相对有限；CNN 通过卷积层和池化层提取局部纹理、边缘和形状特征，测试准确率相比 MLP 有明显提升；ResNet18 在 CNN 的基础上引入残差连接，缓解了深层网络训练过程中的退化问题，进一步增强了模型对高层语义特征的学习能力。在本次实验设置下，MLP、CNN 和 ResNet18 的最终测试准确率分别为 56.67%、83.45% 和 91.92%，其中 ResNet18 取得最优分类效果。实验说明，卷积结构和残差连接能够有效提升图像分类模型的特征表达能力和泛化性能。

关键词：深度学习；图像分类；CIFAR-10；卷积神经网络；ResNet18；PyTorch

第一章 问题定义与理解

1.1 问题定义

本项目的问题定义是：给定一张 32×32 的 CIFAR-10 彩色图像，训练模型从 airplane、automobile、bird、cat、deer、dog、frog、horse、ship、truck 十个类别中预测其所属类别。该任务属于有监督多分类问题，输入为图像像素，输出为类别标签。

随着人工智能和计算机视觉技术的发展，图像分类已经成为模式识别领域的重要任务之一。图像分类的目标是让计算机根据图像内容自动判断其所属类别，在自动驾驶、医学影像分析、安防监控、工业检测和智能相册等场景中具有广泛应用。传统图像分类方法通常依赖人工设计特征，而深度学习模型能够从数据中自动学习多层次特征，因此在图像识别任务中表现出更强的适应能力。

1.2 问题理解

CIFAR-10 图像尺寸较小，但类别之间仍存在明显的形状、纹理和颜色差异。该任务的难点在于图像分辨率低、同类图像姿态变化较大，且部分类别具有相似外观，例如 cat 与 dog、automobile 与 truck。因此，模型不仅需要学习颜色等低级特征，还需要学习边缘、纹理、局部结构和更高层语义特征。

1.3 研究意义

通过 CIFAR-10 图像分类实验，可以系统理解深度学习项目的基本流程，包括数据读取、预处理、模型构建、损失函数设计、优化训练以及分类结果评估。对比 MLP、CNN 与 ResNet18 的性能差异，有助于认识图像空间结构、卷积特征提取和残差连接对分类效果的影响。

1.4 本文工作

本文主要完成以下工作：首先，基于 CIFAR-10 数据集进行图像张量转换、归一化和数据增强；其次，分别实现 MLP、CNN 和 ResNet18 三类模型；然后，在统一训练流程下完成模型训练与测试；最后，结合准确率、损失曲线和混淆矩阵分析不同模型结构的性能差异。

1.5 项目整体流程

本项目从原始 CIFAR-10 数据出发，经过数据预处理、数据增强、模型训练、测试评估和结果保存五个阶段。训练脚本将这些步骤封装为可重复执行的流程，使 MLP、CNN 和 ResNet18 三种模型能够在同一数据集、同一评价指标下进行公平对比。

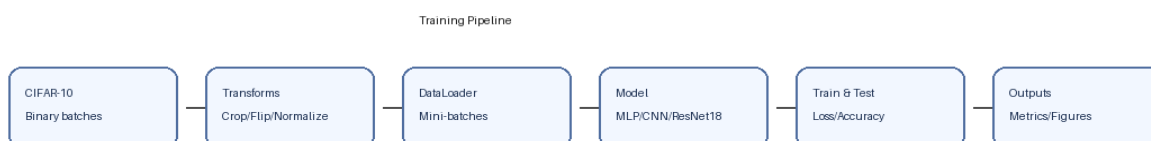


图 1-1 CIFAR-10 图像分类实验整体流程

第二章 相关理论

2.1 神经网络基础

人工神经网络由大量神经元连接构成。每个神经元对输入特征进行加权求和，并通过激活函数引入非线性表达能力。模型训练通常包括前向传播、损失计算和反向传播三个步骤：前向传播得到预测结果，损失函数衡量预测与真实标签之间的差异，反向传播根据梯度更新模型参数。

2.2 卷积神经网络 CNN

卷积神经网络是处理图像数据的经典深度学习结构。卷积层通过局部感受野提取边缘、纹理和形状等局部特征；池化层用于降低特征图尺寸并增强平移不变性；BatchNorm 可以稳定训练过程，Dropout 可降低过拟合风险。相比 MLP，CNN 不需要将图像完全展平，因此能够更好地利用图像的空间结构。

2.3 ResNet 残差网络

随着网络层数增加，深层神经网络可能出现退化问题，即模型更深但训练效果反而下降。ResNet 提出了残差连接结构，使网络学习残差映射而不是直接学习完整映射。残差块中的 shortcut 可以为梯度传播提供更直接的路径，从而缓解深层网络训练困难，使模型能够学习更复杂的视觉特征。

2.4 损失函数与优化方法

本实验采用 CrossEntropyLoss 作为多分类损失函数。对于每张输入图像，模型输出 10 个类别对应的 logits，交叉熵损失会根据真实标签惩罚错误类别概率较高的情况。训练过程中通过反向传播计算梯度，再由优化器更新网络参数。

优化器方面，MLP 和 CNN 使用 AdamW，能够自适应调整不同参数的更新幅度，并通过 weight decay 进行正则化；ResNet18 默认使用 SGD 加动量和余弦退火学习率调度。SGD 在图像分类任务中常能获得较好的泛化性能，而 CosineAnnealingLR 可以让学习率从较大值平滑衰减到较小值，使训练后期更稳定。

第三章 数据分析及处理

3.1 CIFAR-10 数据集分析

CIFAR-10 是常用的小规模彩色图像分类数据集，共包含 60000 张 32×32 RGB 图像，其中训练集 50000 张、测试集 10000 张。数据集包括 10 个类别：airplane、automobile、bird、cat、deer、dog、frog、horse、ship 和 truck，每类样本数量相对均衡。



图 3-1 CIFAR-10 数据集样例图

该数据集采用二进制批文件形式存储，训练程序通过 `torchvision.datasets.CIFAR10` 自动读取并解析为图像张量与标签。由于各类别样本数量均衡，实验中主要关注模型结构和训练策略对分类效果的影响，而不需要额外进行类别重采样处理。

3.2 数据预处理

实验中首先使用 `ToTensor` 将图像转换为 PyTorch 张量，再使用 CIFAR-10 数据集的通道均值和标准差进行 `Normalize` 归一化处理。归一化能够让输入数据分布更加稳定，有助于提升模型训练效率和收敛稳定性。

3.3 数据增强

训练阶段使用 `RandomCrop` 和 `RandomHorizontalFlip` 进行基础数据增强。脚本中还提供 `strong_augment` 选项，可进一步启用 `AutoAugment` 和 `RandomErasing`。数据增强通过生成更丰富的训练样本来提升模型泛化能力，降低模型只记住训练样本细节的风险。

3.4 数据处理代码分析

代码中通过 `build_transforms()` 将多个预处理操作组合起来。训练集和测试集采用不同策略：训练集加入随机裁剪和随机水平翻转以增强泛化能力，测试集只进行张量转换和归一化，保证评估结果稳定。

```
def build_transforms(train, augment, strong_augment):  
    steps = []
```

```
if train and augment:
    steps.extend([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
    ])
    if strong_augment:
        steps.append(transforms.AutoAugment(
            policy=transforms.AutoAugmentPolicy.CIFAR10))
steps.extend([
    transforms.ToTensor(),
    transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),
])
if train and augment and strong_augment:
    steps.append(transforms.RandomErasing(p=0.25))
return transforms.Compose(steps)
```

从代码可以看出，预处理流程具有可配置性。通过 `no-augment` 和 `strong-augment` 参数，可以在基础增强和强增强之间切换，便于观察数据增强对模型性能的影响。

第四章 模型构建

4.1 MLP 模型

MLP 模型首先将 $32 \times 32 \times 3$ 的图像展平成 3072 维向量，然后依次经过多层全连接层、BatchNorm、ReLU 激活函数和 Dropout，最后输出 10 个类别的 logits。该模型结构简单，便于作为基线模型，但展平操作会破坏图像中像素的空间邻接关系，因此对图像局部结构的表达能力较弱。

4.2 CNN 模型

CNN 模型由多组卷积模块组成，每组包含 Conv2d、BatchNorm、ReLU，并通过 MaxPool 或全局平均池化压缩空间维度。该模型能够逐层提取从低级边缘纹理到高级物体部件的特征，分类头使用 Dropout 和 Linear 层输出最终类别。

4.3 ResNet18 模型

ResNet18 是 CNN 的进阶结构，引入残差块和 shortcut 连接以提升深层网络训练效果。由于 CIFAR-10 图像尺寸较小，本文对原始 ResNet18 进行了适配：将第一层卷积改为 3×3 卷积、步长设为 1，并去掉初始 MaxPool 层，从而避免过早损失空间信息。

4.4 核心代码实现分析

本项目的核心训练代码集中在 `train_cifar10.py` 中，整体采用模块化结构实现。代码首先通过 `parse_args()` 读取命令行参数，使模型类型、训练轮数、batch size、学习率、优化器、数据增强和运行设备均可配置；随后通过 `set_seed()` 固定随机种子，减少实验重复运行时的随机波动。

数据处理部分由 `build_transforms()` 和 `build_datasets()` 完成。`build_transforms()` 负责组合 `RandomCrop`、`RandomHorizontalFlip`、`ToTensor`、`Normalize` 等处理步骤，并在需要时启用 `AutoAugment` 与 `RandomErasing`；`build_datasets()` 负责调用 `torchvision.datasets.CIFAR10` 读取训练集和测试集。这样将数据读取与数据增强封装为独立函数，便于后续调整实验配置。

模型构建部分由 `build_model()` 统一调度。MLP、SimpleCNN 和 ResNet18 分别对应三种模型结构，其中 ResNet18 通过 `build_resnet18()` 将首层卷积改为 3×3 、stride 设为 1，并移除初始 MaxPool，以适配 CIFAR-10 的 32×32 小尺寸图像。该处理避免了网络在前几层过快压缩空间分辨率。

训练与评估部分分别由 `train_one_epoch()` 和 `evaluate()` 完成。`train_one_epoch()` 执行前向传播、损失计算、反向传播和参数更新；`evaluate()` 在 `torch.no_grad()` 环境下关闭梯度计算，用于统计测试集 loss、accuracy、预测标签和真实标签。该划分使训练逻辑和测试逻辑相互独立，代码结构更清晰。

核心流程可以概括为：读取参数 → 构建数据集 → 选择模型 → 定义损失函数与优化器 → 循环训练与测试 → 保存最优模型和评估图像。该流程覆盖了深度学习图像分类实验的完整步骤。

4.5 三种模型源代码片段

MLP 的核心是 Flatten 和多层 Linear。它不显式建模像素之间的空间邻域关系，因此适合作为基线模型，用来体现卷积结构带来的性能提升。

```
class MLP(nn.Module):
    def __init__(self, dropout):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(3 * 32 * 32, 1024),
            nn.BatchNorm1d(1024),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(1024, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(512, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, 10),
        )
```

CNN 模型使用多组 Conv2d、BatchNorm、ReLU 和 MaxPool2d，逐层提取局部纹理、边缘和更抽象的物体部件。最后使用 AdaptiveAvgPool2d 将特征图压缩为固定维度，再通过线性层输出类别。

```
class SimpleCNN(nn.Module):
    def __init__(self, dropout):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d((1, 1)),
        )
```

ResNet18 直接调用 torchvision.models.resnet18，并根据 CIFAR-10 小图像特点调整第一层卷积和池化结构。这一改动能保留更多早期空间信息，有利于小尺寸图像分类。

```
def build_resnet18():
    model = models.resnet18(weights=None, num_classes=10)
    model.conv1 = nn.Conv2d(
        3, 64, kernel_size=3, stride=1, padding=1, bias=False)
    model.maxpool = nn.Identity()
    return model
```


第五章 实验设计

5.1 实验环境

项目	配置
操作系统	Windows
Python	3.13.0
深度学习框架	PyTorch 2.12.0+cu126
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
数据集	CIFAR-10

5.2 超参数设置

参数	数值
Epoch	20
Batch Size	128
Learning Rate	MLP/CNN: 0.001; ResNet18: 自动采用 0.1
Optimizer	MLP/CNN: AdamW; ResNet18: SGD (自动适配)
Weight Decay	MLP/CNN: 1e-4; ResNet18: 5e-4
Loss	CrossEntropyLoss (label smoothing=0.05)

5.3 评价指标

本文主要采用 Accuracy、Loss 和 Confusion Matrix 进行评价。Accuracy 用于衡量模型整体分类正确率；Loss 用于观察模型收敛情况；混淆矩阵用于分析不同类别之间的误分类关系。

5.4 训练流程与参数控制

实验训练采用 CrossEntropyLoss 作为分类损失函数，并支持 label smoothing 来缓解模型过度自信的问题。优化器方面，普通 MLP 和 CNN 默认使用 AdamW，ResNet18 在默认参数下会自动切换为 SGD，并使用较大的初始学习率和 weight decay，以便在 20 个 epoch 内获得更好的收敛效果。

学习率调度采用 CosineAnnealingLR，使学习率随训练轮数逐渐下降。相比固定学习率，余弦退火可以在训练前期保持较强的参数更新能力，在后期减小更新幅度，帮助模型稳定收敛。每个 epoch 后，程序都会在测试集上评估模型，并根据测试准确率保存 best_model.pt。

为了提升代码鲁棒性,脚本提供 `device=auto` 模式:当检测到 CUDA 可用时自动使用 GPU,否则回退到 CPU。同时,`subset-size` 参数可以在算力有限时抽取部分训练数据进行快速实验,`smoke-test` 参数可以使用 FakeData 快速检查代码流程是否可运行。

5.5 训练与评估代码分析

训练函数 `train_one_epoch()` 体现了深度学习训练的标准流程:将数据送入设备,清空梯度,前向传播得到 logits,计算损失,反向传播,再调用 `optimizer.step()` 更新参数。评估函数 `evaluate()` 则关闭梯度计算,只统计 loss、accuracy 和预测标签。

```
for images, labels in loader:
    images = images.to(device)
    labels = labels.to(device)

    optimizer.zero_grad(set_to_none=True)
    logits = model(images)
    loss = criterion(logits, labels)
    loss.backward()
    optimizer.step()

    correct += (logits.argmax(dim=1) == labels).sum().item()
```

`compare_models.py` 通过 `subprocess` 依次调用训练脚本,使三种模型复用同一套训练和评估代码,减少手动操作带来的误差。训练结束后,脚本读取每个模型最新的 `metrics.json`,并生成 `comparison_summary.json`。

```
for model in ("mlp", "cnn", "resnet18"):
    metrics = run_model(args, model)
    summary[model] = {
        "best_test_accuracy": metrics["best_test_accuracy"],
        "final_test_accuracy": metrics["final_test_accuracy"],
        "final_test_loss": metrics["final_test_loss"],
    }
```

第六章 实验结果与分析

6.1 Loss 与 Accuracy 曲线分析

从训练曲线可以看出，随着训练轮数增加，三个模型的损失总体下降，测试准确率逐步提升，说明模型能够从训练数据中学习有效特征。MLP 的提升速度和最终效果相对有限；CNN 的测试准确率明显高于 MLP；ResNet18 在中后期仍能继续提升，表现出更强的特征表达能力。

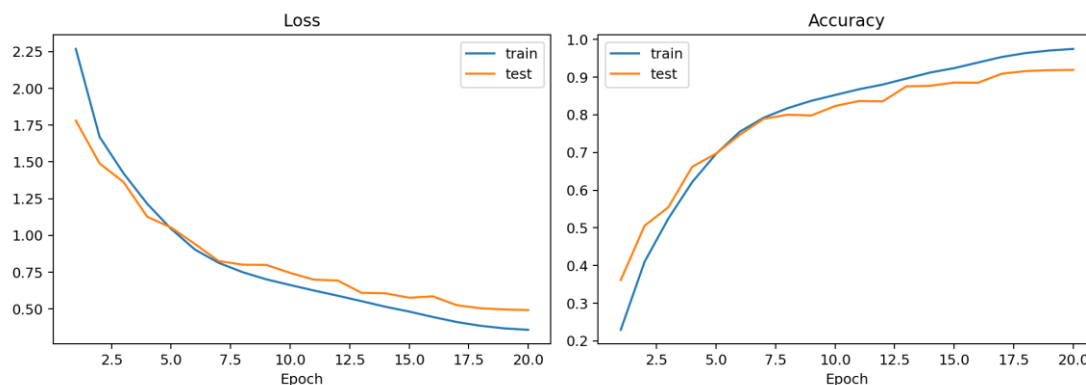


图 6-1 ResNet18 训练/测试损失与准确率曲线

6.2 混淆矩阵分析

混淆矩阵能够展示每个真实类别被预测为不同类别的情况。对于 CIFAR-10，cat 与 dog、truck 与 automobile 等类别在外观上存在一定相似性，因此更容易出现混淆。ResNet18 的整体混淆情况较少，说明残差网络对细粒度视觉特征具有更好的学习能力。

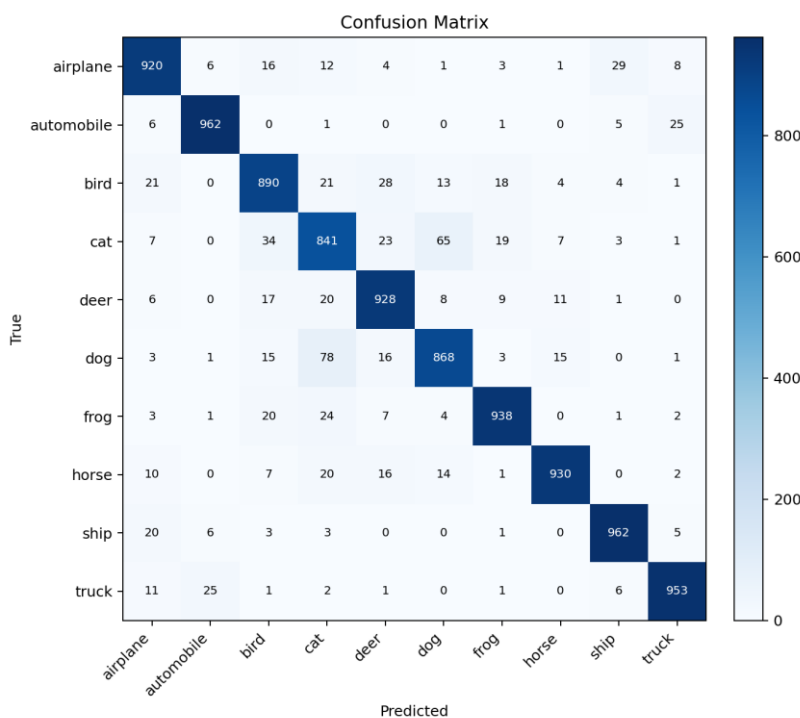


图 6-2 ResNet18 测试集混淆矩阵

6.3 不同模型性能对比

模型	最佳测试准确率	最终测试准确率	最终测试损失
MLP	56.67%	56.67%	1.3389
CNN	83.45%	83.45%	0.7562
ResNet18	91.92%	91.92%	0.4924

实验结果显示，MLP 的最终测试准确率为 56.67%，CNN 提升到 83.45%，ResNet18 达到 91.92%。这说明模型结构越能利用图像空间结构和深层语义特征，分类性能越好。MLP 只能基于展平后的像素向量进行学习；CNN 利用卷积核提取局部特征；ResNet18 则进一步通过残差结构训练更深层网络，因此取得最优表现。

从分类报告看，ResNet18 在 automobile、ship、truck 等类别上的 F1-score 较高，而 cat 和 dog 等类别相对更难区分。这与自然图像中动物类别外观差异较小、姿态变化较大的特点一致。

6.4 三种模型可视化结果对比

为了更全面地观察不同模型的训练过程和分类错误分布，本文将 MLP、CNN 和 ResNet18 的训练曲线与混淆矩阵均纳入结果分析。训练曲线展示模型收敛趋势，混淆矩阵展示不同类别之间的误分类情况。

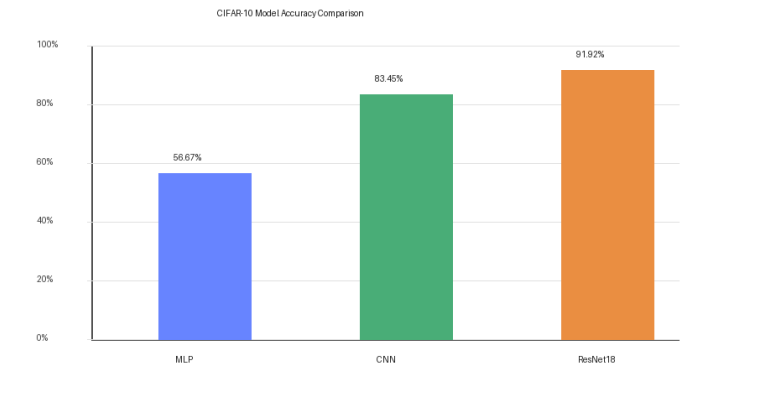


图 6-3 三种模型测试准确率对比

MLP 模型结果分析

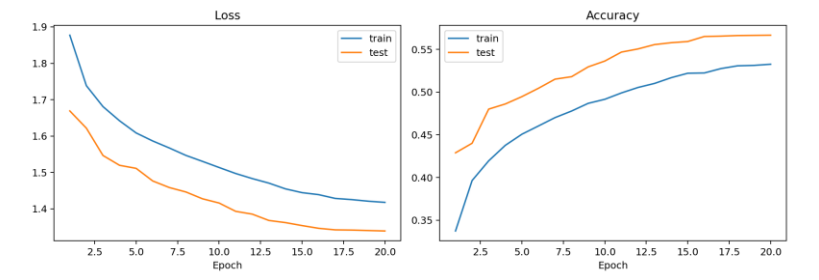


图 6-4 MLP 训练/测试损失与准确率曲线

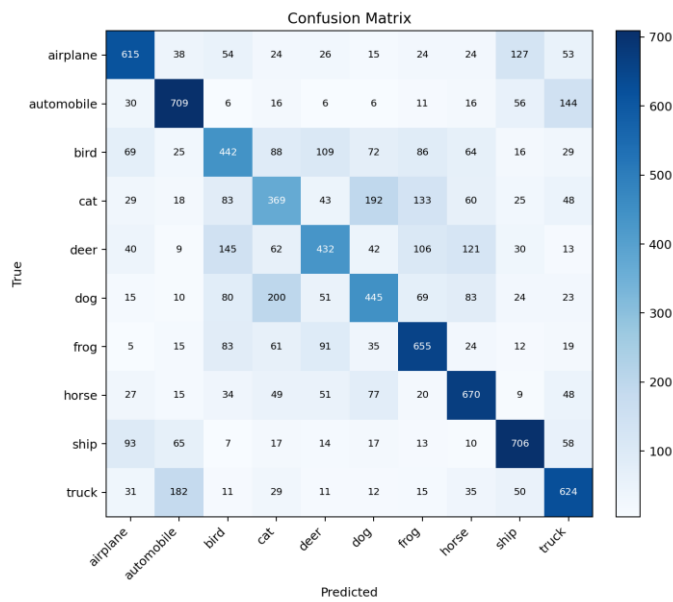


图 6-5 MLP 测试集混淆矩阵

MLP 的准确率提升较慢，最终测试准确率为 56.67%。其主要原因是展平操作破坏了图像空间结构，模型难以捕捉局部边缘、纹理和形状关系。

CNN 模型结果分析

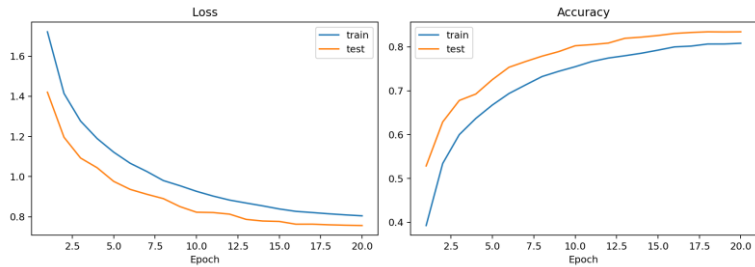


图 6-6 CNN 训练/测试损失与准确率曲线

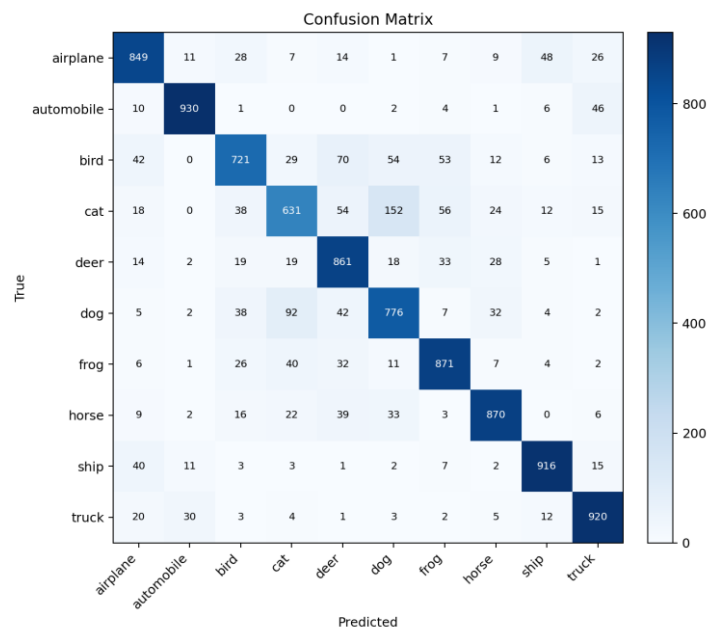


图 6-7 CNN 测试集混淆矩阵

CNN 的测试准确率达到 83.45%，相比 MLP 有明显提升。卷积层能够共享参数并提取局部视觉特征，因此更适合处理图像数据。

ResNet18 模型结果分析

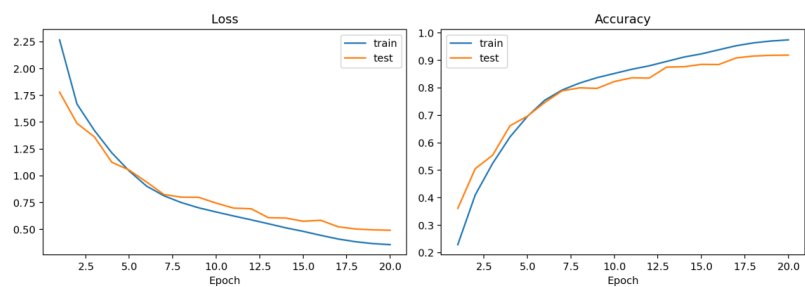


图 6-8 ResNet18 训练/测试损失与准确率曲线

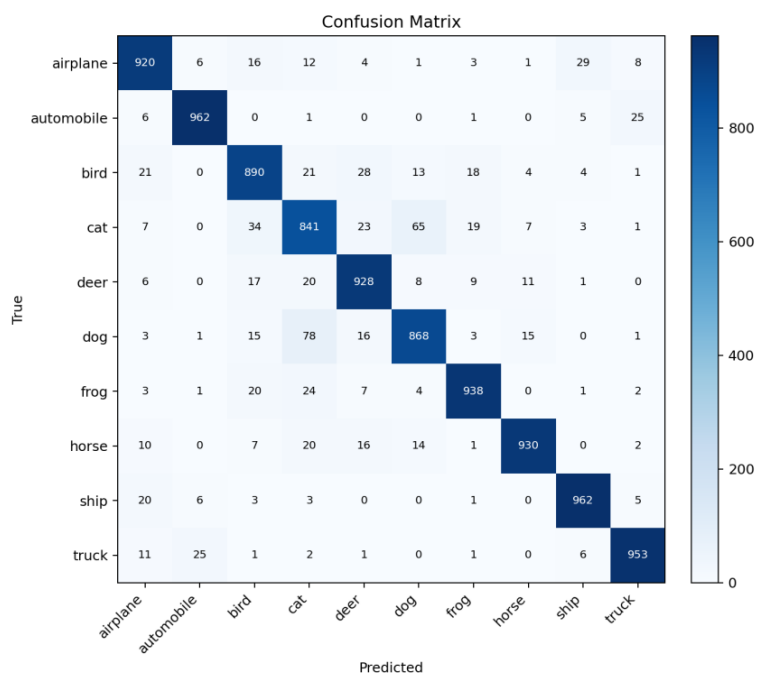


图 6-9 ResNet18 测试集混淆矩阵

ResNet18 的测试准确率达到 91.92%，在三种模型中表现最好。残差连接缓解了深层网络训练难度，使模型能够学习更丰富的高层语义特征。

6.5 结果文件与可视化输出

每次训练会在 `runs` 目录下生成一个以时间和模型名称命名的实验文件夹。该文件夹中包含 `best_model.pt`、`metrics.json`、`loss_accuracy_curves.png` 和 `confusion_matrix.png`。其中 `best_model.pt` 保存测试集准确率最高的模型权重，`metrics.json` 保存训练参数、最终准确率、分类报告和每轮历史指标。

`compare_models.py` 会依次调用 `train_cifar10.py` 训练 MLP、CNN 和 ResNet18，并将三种模型的最终结果汇总到 `runs/comparison_summary.json`。这样既保留了每个模型的详细结果，又能直接得到模型间的横向对比，方便在报告中分析模型结构差异带来的性能变化。

从输出结果看，MLP 的准确率较低，说明仅依靠全连接层难以充分表达图像空间结构；CNN 显著提升准确率，说明卷积操作有效提取了局部视觉特征；ResNet18 进一步提升到 91.92%，说明残差连接和更深层网络有助于学习更复杂的图像语义特征。

第七章 结论与可能改进

本文基于 CIFAR-10 数据集完成了图像分类实验，构建并比较了 MLP、CNN 和 ResNet18 三种模型。实验结果表明，CNN 相比 MLP 更适合处理图像数据，而 ResNet18 通过残差结构进一步提升了分类性能。整体来看，模型性能呈现 $MLP < CNN < ResNet18$ 的趋势，说明卷积结构和深层残差连接对图像分类任务具有重要作用。

后续工作可以从以下方向继续改进：使用更深的 ResNet 或 WideResNet；引入迁移学习和预训练模型；尝试 Vision Transformer 等新型视觉模型；进一步调节数据增强、学习率调度和正则化策略，以提升模型泛化能力。

7.1 主要结论

综合实验结果可以得出三点结论。第一，图像分类任务需要充分利用空间结构，单纯的全连接网络难以获得较高准确率。第二，卷积神经网络通过局部连接和参数共享显著提升了特征提取能力。第三，ResNet18 在普通 CNN 基础上引入残差连接，进一步提升了深层网络训练效果，是本实验中性能最优的模型。

7.2 可能改进

后续可以从四个方向继续改进：一是使用更强的网络结构，例如 WideResNet、DenseNet 或 EfficientNet；二是引入迁移学习，使用 ImageNet 预训练权重提升收敛速度；三是进一步调节数据增强、学习率和正则化策略；四是增加多次重复实验，统计平均准确率和方差，使实验结论更加稳定可靠。

参考文献

- [1] Krizhevsky A. Learning Multiple Layers of Features from Tiny Images[R]. 2009.
- [2] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998.
- [3] He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[C]. CVPR, 2016.
- [4] PyTorch Documentation. <https://pytorch.org/docs/>
- [5] CIFAR-10 Dataset. <https://www.cs.toronto.edu/~kriz/cifar.html>

独立性声明

本人郑重声明：本报告内容是由作者本人独立完成的。有关观点、方法、数据和文献等的引用已在文中指出。除文中已注明引用的内容外，本报告不包含任何其他个人或集体已经公开发表的作品成果，不存在剽窃、抄袭行为。

特此声明！

作者签名：__庄梓泓__

日期：__2026__年__5__月__18__日